

**This dissertation has been
microfilmed exactly as received**

70-3806

**KAY, Alan Curtis, 1940-
THE REACTIVE ENGINE.**

**University of Utah, Ph.D., 1969
Computer Science**

University Microfilms, Inc., Ann Arbor, Michigan

© Alan Curtis Kay 1970

ALL RIGHTS RESERVED

PREVIEW

THE REACTIVE ENGINE

by

Alan Curtis Kay

A thesis submitted to the faculty of the
University of Utah in partial fulfillment of the
requirements for the degree of

Doctor of Philosophy

Department of Electrical Engineering
Computer Science

University of Utah
August 1969

This Thesis for the
Doctor of Philosophy Degree

by

Alan C. Kay

has been approved
July 1969

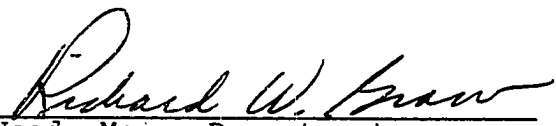

Chairman, Supervisory Committee


Reader, Supervisory Committee


Reader, Supervisory Committee


Reader, Supervisory Committee


Reader, Supervisory Committee

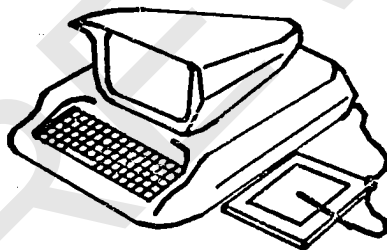

Head, Major Department


Dean, Graduate School

THE REACTIVE ENGINE

by

Alan C. Kay



"I wish to God these calculations
were executed by steam."

-- C. Babbage
The Analytical Engine

Acknowledgements

This work could not have prospered without the challenging and pertinent criticisms of many people and the catalytic nature of the environment in the Computer Science Department of the University of Utah.

I would particularly like to thank Professors David C. Evans and Robert S. Barton for supplying the motivation, Patricia M. Madsen for providing the inspiration, and Edward C. Cheadle for making it all possible.

I also wish to thank C. Stephen Carr for many hours of fascinating discussion that certainly have had a lot to do with the present form of this work.

Table of Contents

Acknowledgements	iv
Abstract	vii
I. a. Preface	1
b. Reading Grade	4
II. Part A: The Basic Dialogue	6
1. Introduction	7
2. A Simple Interactive Language Interpreter	43
3. The FLEX Language	69
a. Translation	77
b. Deferment and Coercions	87
c. Abstraction and Attributes	98
d. Elaboration	110
e. Mapping	119
f. User Interaction	127
g. Modeling	133
h. An NC Language Proposal and Realization	143
III. Part B: Physical Considerations	176
4. The Pragmatic Environment	177
5. Segments and Processes	182
6. Memory Systems	192
a. Mapping	
i. Contiguous Mapping	193
ii. Paged Segments	200
iii. Multi-level Schemes	205
b. Secondary Memory	
i. Swapping	214
ii. The Disk as a Serial "Associative" Memory	218
c. An Associatively Mapped LSI Memory	223
7. Algorithmic Processing	227
8. The Display Processor	233
9. Input and Output	251
10. Multi-machine Configurations	255

IV. Implementations to Date	262
V. Appendix A. The FLEX Handbook	264
References	320
Vita	327

PREVIEW

Abstract

The design of machines which can participate in an interactive dialogue is the main topic of this thesis. To do this successfully, a language processor and a more general memory system will be incorporated as the lowest level of the hardware.

A higher level kernel language, FLEX, which is comprehensive enough to provide the basic dialogue, extend itself into more desirable forms, and describe other languages, yet is concise enough to be implemented directly as a hardware processor, is developed.

Several models of abstract interpreters for this reactive system, starting with a very simple textual model, are then discussed, followed by hardware representations of these schemes.

These considerations lead to the design and implementation of the "FLEX Machine", a personal, reactive, mini-computer which communicates in text and pictures by means of keyboard, line-drawing CRT, and tablet.

"Maybe Computer Science
should be in the College of
Theology!"

-- R. S. Barton
1967

I. Preface

A rationale for writing a dissertation (besides, the obvious one) should have something to do with the desire to shorten someone else's path to a given point so that he may press on to more interesting horizons. For that reason, an attempt has been made to write a somewhat verbose, but rather complete, account of the past two year's work. Since this paper involves the entire design of a computer system: "hard-", "firm-", and "software", the considerations leading to some of the decisions and compromises must, of necessity, be somewhat involved if one is to "tell all". Also, it is strongly felt that the crux points of decision making add more to the philosophical nature of a work than the rather concise engineering presentation to be found in Appendix A.

Granted, it is nice that a complete design, because of its inherent recursivity, may be boiled down to a few pages of symbols -- for that is the nature and purpose of abstraction. Yet also, whenever a work such as this has been completed, it should be relegated immediately to the novice information scientist so that it need never again seem erudite. It is to this reader that "The Reactive Engine" is dedicated, not to the so-called Computer Science professional.

Thus, a low level (but accelerated) approach has been taken. It is impossible to present all of the introductory material that is necessary for the understanding of this thesis in one short chapter. Nevertheless, that chapter is most important to the novice reader, since it seeks to provide an intuitive feel for the environmental concepts and (more to the point) tries to instill motivation for consulting the basic references in the field.

Because there is a very real difference between the form and effects experienced by an interactive user of the FLEX environment and the quibblings and rationalizations about why things turned out as they did. This paper has been divided into a number of documents.

This portion contains the considerations and philosophy leading to the gross design of an interactive mechanism. An informal description of the external aspects, followed by a formal definition of the entire machine, is found in "The FLEX Handbook", which can also serve as a user's manual. The other section consists of large charts which it is convenient to spread out separately.

There seems to be an interesting sociological connection between the rise and acceptance of the new dogma, as opposed to the old. The Pythoness at Delphi gagging on the fumes of insight has nothing on her modern counterparts, who are overwhelmed by a suffocating flow of bits. The quotations at the chapter heads, gathered from conversations (and

certain sacred and profane documents), reflect the basic duality of Computer Science (and human nature).

Alan C. Kay
Salt Lake City
August , 1969

PREVIEW

I.b. A Reader's Guide to Non-Periodic Literature

This work could be subtitled: "A Designer's View". As such, its extreme length is somewhat intimidating to the casual reader, though it may be welcomed by the more serious student. This sub-preface will discuss a number of possible paths through the document.

Although each section builds on the preceding sections, the paper need not be read in a serial order. Sections II.1 and II.2 are included primarily as a philosophic base to the rest of the book and may be skimmed or skipped on first reading by the pragmatic reader. II.3 provides a reasonable starting place for the "Cooks Tour". II.3h on the numerically-controlled tool system is a rather complete, somewhat independent paper which may be omitted on first reading.

Those readers familiar with mapping schemes and storage allocation problems may omit reading most of III.6. They may find III.6bii and III.6i to be somewhat novel.

Sections III.9 and III.10 are not central to the issues of this work and will not stunt the reader's growth if he passes over them.

Appendix A: The FLEX Handbook, may be read independently of the rest of the text. It is in the form of a user's manual and only presents "how's", not "why's".

The comprehensive reader will discover that, although all sections relate to the FLEX language and machine, the book itself may be regarded as a set of somewhat independent

papers covering many criteria of machine design and, hence, may be approached in that fashion.

PREVIEW

II. Part A: The Basic Dialogue

"then in certain cases and for irresponsible men it may be that non-existent things can be described more easily and with less responsibility in words than the existent, and therefore the reverse applies for pious and scholarly historians: for nothing destroys description so much as words, and yet there is nothing more necessary than to place before the eyes of men certain things the existence of which is neither provable nor probable, but which, for this very reason, pious and scholarly men treat to a certain extent as existent in order that they may be led a step further towards their being and their becoming."

--Joseph Knecht's holograph
translation (M.L.)

"Systems programmers are high
priests of a low cult."

--R. S. Barton
1967

1. Introduction

It is not the purpose of this book to determine why human beings build machines, nor is it to describe how. Yet a certain amount of metaphysics may creep into the discussion from time to time, since any investigation into the design of devices which are actually to be used by people must of necessity try to guess what the class of potential users will want. Of course, they do not know themselves if their needs are potentially interesting, and it is useless to poll the remainder (as previous experiments have shown) for they exhibit the well-known "committee syndrome": "...the only form of life with ten bellies and no brain."

A good class project for undergraduates who have not become too tainted with either the commercial or research computing milieu, is to have them design a computer system for a "think tank" such as RAND or the Institute for Advanced Study at Princeton. It is a delightfully nebulous question, since they quickly realize it will be impossible for them to even discover what the majority of the thinkers are doing. Indeed, many of the researchers will not know themselves or be able to articulate that state of mind of "just feeling around". It is at this point that a wide

philosophical division appears in the students. Almost all of them agree that there is really nothing that they can do for the scientists. The more engineering-minded of the students, seeing no hard and fast solution, stop there. The rest, who are somewhat more fanciful in their thoughts, say "...maybe 'nothing' is exactly the right thing to deliver, providing it is served up in the proper package". They have articulated an important thought. Not being able to solve any one scientist's problems, they nevertheless feel that they can provide tools in which the "thinker" can describe his own solutions and that these tools need not treat specifically any given area of discourse.

The latter group of students has come to look at a computing engine not as a device to solve differential equations, nor to process data in any given way, but rather as an abstraction of a well-defined universe which may resemble other well-known universes to any necessary degree. When viewed from this vantage point, it is seen that some models may be less interesting than the basic machine (payroll programs). Others may be more interesting (simulation of new designs, etc.). Finally, when they notice that the power of modeling can extend to simulate a communications network, an entirely new direction for providing a system is suggested.

While they may not know the jargon and models of an abstruse field, yet possibly enough in general of human

communications is known for a meta-system to be created in which the specialist himself may describe the symbol system necessary to his work. In order for a tool such as this to be useful, several constraints must be true.

1. The communications device must be as available (in every way) as a slide rule.
2. The service must not be esoteric to use. (It must be learnable in private.)
3. The transactions must inspire confidence. ("Kindness" should be an integral part.)

The systems described in this book do not claim to completely satisfy these conditions, but they have definitely used these considerations as a global basis for the entire project.

First Principles

Machines which do one thing only are boring, yet exert a terrible fascination. The black box, whose only action when switched on is to exude a hand which turns the switch to "off", is a popular symbol of the "futility of it all". It was the creation of machines with exactly the opposite intent which got the whole business started.

In 1801, J. M. Jacquard decided that looms which could only weave one pattern were unbearably dull and devised a parametric drawloom which was "driven" by punched cards. Babbage owned what was probably the first instance of "computer graphics", a woven portrait of Jacquard in silk done

on the parametric loom whose resolution (1000 threads to the linear inch) approached that of the very best devices today. It was this notion of an interpreter basing its decisions on stored parameters that so fired Babbage's imagination. Unfortunately, neither the technology nor the mentality of his century were fertile ground for his ideas.

Of course, modeling systems have existed for as long as the human species. In fact, some anthropologists base their partition of primates into sapient and non-sapient beings solely on the existence of a well-developed language. It is well known that humans need to build a mental model of their environment before they can apprehend it in any way. Probably the two greatest discoveries to be made were the importance of position in a series of grunts, and that one grunt could abbreviate a series of grunts. Those two principles, called syntax (or form) and abstraction, are the essence of this work.

It is fascinating to note the relationship between the form of languages and the way a culture looks at the world.

The basic framework of the Indo-European languages is that of the verb inflected for tense, etc., which closely determines the way abstractions are partitioned into sounds. To this speaker, the world has objects which endure over time. Verbs connote actions and processes. Adjectives

indicate permanent qualities, while transitory qualities (and qualities of qualities) are handled by several types of adverbs. Causality is tied to the ordering imposed by tense: producing "effect". Things are discrete, etc.

This is a very digital world and much of it has counterparts in the basic architecture of present-day computers. However, much of mathematics describes another, more continuous world. "Cause" and "effect" are functional in nature, and time is viewed from a God-like vantage point. In this framework, there are relations between notions rather than discrete actions. Interestingly enough, there are human languages which reflect this very different way of apprehending the world. It was the discovery of Hopi that led Whorf (1) to the conclusion that the way a group of people see, hear and think about things is constrained by their language, since that is actually the medium through which they build and manipulate their internal models of the world.

Noting these thoughts leads one to a sobering situation when contemplating the design of a system to represent the form (syntax) and the modeling (semantics) that is necessary to represent a universe (albeit a small one). These weighty considerations will be treated in a very "engineering" and pragmatic way: they will be always remembered, adhered to when possible, and ignored when necessary!

Concepts and Vocabulary

In keeping with the principle of abstraction, it will be much simpler to define some terms which will be used frequently rather than to continually refer to these notions by circumlocutions. An intuitive (rather than axiomatic) approach is taken, with the belief that it is possible to be precise without being stuffy. Necessary formalities are to be found in the appendices and the references; they should not be sought here.

A few simple concepts will suffice to open up this investigation.

Sets

A set is thought to be a collection of "things" and may be represented by "{", "}" surrounding the elements that are considered to be members.

Ordered Pairs

This fundamental notion will be symbolized by angle brackets with a dot separating the pair. $\langle x.y \rangle$ is an ordered pair where x and y may themselves be ordered pairs, etc.

Strings

For now, a string is a set whose members are ordered. For example, the word, "string", consists of the unordered set $\{ "t", "i", "n", "s", "g", "r" \}$ of letters and an ordering giving $\langle "s", "t", "r", "i", "n", "g" \rangle$. An alternative

definition defines "&" as the concatenation operator, such that "a" & "b" is the string "ab". So "string" may be formed by "s" & "t" & "r" & "i" & "n" & "g" and will be assumed to be equivalent to the angle bracket notation until further elaboration.

Names

A name is a string starting with a letter and followed by an arbitrary sequence of letters or digits, or a string starting with a digit, followed by an arbitrary sequence of digits.

Free and Bound Names

A name as defined above consists solely of a sequence of characters. To paraphrase Humpty Dumpty: "It means just what you want it to mean, nothing more nor less". As such, it is a very unsatisfactory creature and is roughly equivalent in semantic content to an inelegant simian belch. Consider now a sequence of digits: 1245. Ink marks on a page! It has the same vacuous extent as the name above. But now you say, "I know those marks mean a set with one thousand twelve hundred and forty-five members! Not so, for computer scientists are not to be trusted. The author may exert his "Dumptyrian" prerogatives to assert that those marks actually represent something in base_8; i.e., the set that has 677 (base_10) members. Sequences of characters about which only their textual representation is

known are considered to be "free" (and slippery) names.

Names may be "bound" down by specifying some:

Property Names and Referents

The representation: $\text{base}_8 \bullet 1245 \equiv \{1, 1, 1 \dots, 1\}$ relates two names and a set using two other marks for clarification. $\bullet$ may be read "of". $\equiv$ may be read "is defined to be". However, these English "equivalents" do not clear the air. What truly matters is that associated with the name "1245" there is an ordered pair: $\langle \text{base}_8 \bullet \{1, 1, 1 \dots, 1\} \rangle$

The first member may be called the property name or attribute, the second, the referent or the value. This pair represents a binding of the name 1245. Another binding would be:

$$\langle \text{base}_{10} \bullet \{1, 1, 1 \dots, 1\} \rangle$$

where the number of members of the referent is presumably different. Now it can be stated (with tongue in cheek) that the things that are "known" about 1245 can be expressed as the set of ordered pairs:

$$1245: \{ \langle \text{"base}_8" \bullet \{1, 1 \dots, 1\} \rangle, \langle \text{"base}_{10}" \bullet \{1, 1, 1, \dots, 1\} \rangle \}$$

In algebra, when it is wished for x to "stand for" the value 5, expressed as $x \leftarrow 5$, it is

value • $x \equiv 5$ that is meant.

This may seem like quibbling, but it is very convenient to be able to nail down the bindings of "x" when challenged. There may be other bindings which are apparent to the reader, such as

"style" • $x = \textcircled{C}$ IBM Selectric: "Delegate".

Circumlocutions

"The person who, through consummate artistry and careful application, seasons and arranges the products of those who toil on the soil and in the plains" is a circumlocution for "chef". Languages which do not have a ready mechanism for replacing the longer form with the short have grave difficulty in transmitting complicated constructions. A number of American Indian languages have this problem and, while it is quite possible to express any concept by means of a circumlocution, the system is not well suited for human communication because of the short attention span of the listener. The set with 677 (base_10) numbers is a rather uncouth circumlocution for base_8 • 1245. Indeed, the author cheated by sneaking in an elipsis of three dots (...) to signify "lots" because he didn't feel that it was necessary to exhibit the entire set. And that, indeed, is the crux of this matter. It truly is rarely necessary to exhibit all of more complicated constructs to deal with them

successfully. The Trobriand Islander gets along well with:

```

base_10 • 1 = {1}
base_10 • 2 = {1, 1}
base_10 • 3 = {1, 1, 1}
base_10 • 4 = "many"
      ⋮
base_10 • 1245 = "many"

```

Other Ordered Pairs

So far the bindings of names has been expressed as sets of ordered pairs establishing relations. It should be noted that other pairs may be formed which can express equivalent "meanings". The intension of the attribute may also be expressed as a set of ordered pairs. The following two representations may be considered to be equivalent:

$$\begin{array}{ll}
 \leq \bullet 5 \equiv 4 & \text{gives } 4 : \langle \leq \bullet 3 \rangle \\
 \leq \bullet 4 \equiv 3 & \text{and } 5 : \langle \leq \bullet \{4, 3\} \rangle \\
 \leq \bullet 5 \equiv 3 & \text{or } \leq : \{ \langle 5 \bullet 4 \rangle , \langle 4 \bullet 3 \rangle \}
 \end{array}$$

The latter representation is the way relations are defined in mathematical logic: as the set of ordered pairs exhibiting all the possibilities.

The word function can be taken to mean a relation defined as a set of ordered pairs. If $\leq$ is thought to be a function and defined for 5: $\leq \bullet 5$, one would hope that the set $\{3, 4\}$ would be exhibited as the possible right-hand members of $\langle 5 \bullet 3 \rangle$ and $\langle 5 \bullet 4 \rangle$. Note that $\langle 5 \bullet 3 \rangle$ and $\langle 5 \bullet 4 \rangle$ are equivalent to $\langle 5 \bullet \{3, 4\} \rangle$ in this formulation.